Original Contribution

WebGL Rendering: Using Physical-Based Materials and Lighting to Create a Virtual Environment on the Web

Karu Lal[1]✉

Keywords: WebGL, Modelling, Interface, 3D Web, Lighting, Performance

Asian Journal of Applied Science and Engineering

Vol. 11, Issue 1, 2022 [Pages 56-64]

The environment of the World Wide Web is increasingly becoming more significant for incorporating three-dimensional content. Since the introduction of WebGL to the present day, many solutions (including libraries, middleware, and apps) have been developed, encouraging the development of 3D data as a kind of practical online media. Because of the rapid advancement of 3D technology and other associated web-based resources, it isn't easy to recognize and correctly comprehend the most recent trends and open questions in the industry. This survey analyzes the current state of the art of 3D web publishing, reviews the possibilities provided by the primary current approaches, and categorizes the features supported by existing solutions, and cross-maps these with the requirements of a few main application domains. The survey begins with these premises and examines the state of the art of 3D web publishing. The findings of this investigation help define the technical features required to construct an efficient and effective 3D data presentation while considering the various application settings.

INTRODUCTION

Three-dimensional (3D) data has grown from specialized material utilized by a limited community of professionals to a fully integrated web medium that is mature. Although the technology to support this new media has been available for years, users' perspectives have only lately evolved, allowing 3D web content to reach the larger public (Amin & Mandapuram, 2021). The democratization of 3D content creation (low-cost 3D scanning devices, improvement of 3D-from-images/structure-from-motion approaches, consolidation of manual modeling systems) and the introduction of game-changing contributions for a broader range of target users (3D printing applications, 3D viewer and editing systems embedded in standard operating systems, etc.) were crucial to this evolutionary process (Reddy et al., 2020).

These new developments propel 3D content into an unknown world of data management, user interactions, and cross-media integration. Our proposed ecosystem is part of the web, a democratic space of excellence. Recently, there has been increasing focus on integrating three-dimensional material and developing tools for this purpose (Mandapuram & Hosen, 2018). While 3D content has gained popularity in recent years, attempts to deliver it online also have a long history. Web developers and 3D professionals soon saw the potential of extending the web to 3D data; therefore, 3D should not be limited to standalone applications (Sun et al., 2015).

The rapid start differed from developing practical and consistent solutions, leading to a lengthy and winding journey towards an effective Web3D. Pioneering landmarks include the 1996 Macromedia Flash plugin, the first approach to handling online interactive multimedia content, and the Apple Webkit CANVAS, the first JavaScript-controlled HTML drawing element. The web has been dominated by proprietary systems, third-party applications, and closed solutions for years

[1]Integration Engineer, Ohio National Financial Services, USA
✉Corresponding email: karu.lal84@gmail.com

(Deming et al., 2018). Lack of a uniform development standard limited web 3D content publication and utilization.

The advent of the WebGL application programming interface (API) was a huge milestone, launching a new generation of apps based on a common standard that could directly affect the rendering pipeline and be supported by all major web browsers. WebGL put Web3D in a new age. The first assessment on web-based 3D graphics shows its maturity four years after WebGL's inception. Current Web3D techniques, including academic and commercial systems, vary in data presentation strategies based on 3D content, target consumers, publishing venue, application field, and desired output. The prevalence of 3D solutions on the web has led to a complex situation where developers and users need help navigating, particularly those with a limited understanding of the unique needs of each 3D data and application domain combination.

This poll reviews options for meeting these needs. The primary objective is to create a schema of the features and capabilities enabled by the supporting technologies and systems (Desamsetti, 2021). This map aims to guide readers through technical characteristics for efficient and effective Web3D presentations, depending on the application field. This survey seeks to aid readers in understanding the various phases of 3D publishing, including content creators, content consumers, and future solution researchers and developers. This review examines a variety of software programs and the current scientific literature. Characterizing existing solutions took time due to the diversity of approaches and the number of factors to study. This survey emphasizes current trends and obstacles in efficiently transferring advanced 3D visuals to the web for researchers and developers (Sawicki & Chaber, 2013).

WEB INTERFACE FOR 3D MODELING AND SIMULATION USING WEBGL

Web-based platforms are widely adopted in today's engineering applications across various domains to do interactive 3D model-based visual simulations. Any web browser that supports HTML5 can render 2D or 3D graphics using WebGL, an application programming interface written in JavaScript (Desamsetti, 2016b). It uses the canvas element introduced with HTML5 and can be accessed using the document object model APIs.The WebGL renderer module makes it possible to produce any 2D or 3D graphics within the canvas without any other plugins. These visuals may be seen in any standard web browser compatible with HTML5. When constructing any 3D virtual model, it is possible to quickly and easily design simple geometries by defining the data associated with the geometrical vertex points. After the preliminary designing phase has been completed with CAD software such as Solidworks, CATIA, Autodesk, Creo, and so on, the extracted triangular vertex coordinate data are merged into the WebGL program. This step is performed for complex three-dimensional components. Applying direct usage of the WebGL program is a good technique; however, it leads to difficulties when attempting to draw a three-dimensional structure comprising several components. One must correctly specify all vertex coordinates data points to visualize a single component. These data points must be extracted using a tool or algorithm from the CAD model. Because of this, making direct use of the WebGL program can be laborious, whereas utilizing a library can shorten the number of steps involved and give a more natural means of generating 3D geometric models (Dekkati & Thaduri, 2017). At the moment, numerous JavaScript libraries such as Three.js, SceneJS, and BabylonJS employ WebGL and provide a great deal of high-level functionality for the visualization of 3D objects.Three.js is a high-level utility package that simplifies and streamlines the process of programming with WebGL. It is an open-source JavaScript 3D framework that uses WebGL technology and offers a variety of capabilities, including control over the camera and the lighting, as well as animations (Thaduri, 2017). Because it is coded in JavaScript, it may be integrated with other libraries to produce interactive apps. This is made possible by the language's versatility (Dekkati et al., 2019). In many different areas of study, the application of this technology to conduct various web-based experiments is currently in progress. In this instance, the Three.js package is utilized for 3D modeling and specific scene visualization objectives. Now, depending on the amount of difficulty of the 3D model that needs to be generated, various approaches to its creation can be utilized, which will be detailed in the next part (Kang & Lee, 2017).

3D MODELING FROM BASIC SHAPES/GEOMETRIES

To begin the process of 3D modeling through the web environment, an initial set of chores needs to be completed. To display a 3D model on a web browser, the first thing that needs to be done is to create an HTML5 canvas within which the scene will be defined using THREE. The Scene function and the camera are set, after which their attributes are specified (Ballamudi, 2016). The scene functions as a container that stores all the items that will be rendered. After that, the THREE.-WebGLRenderer method calls the WebGL renderer module, which is responsible for causing the scene and the camera. To generate any conventional 3D shape with the help of the three.js library and visualize it within the

canvas, the following steps in sequential order need to be followed, as demonstrated (Desamsetti, 2020). The first thing that must be done to model any 3D structure is to create the basic geometry of a single construction block (Thaduri, 2018). After the thing's dimensions have been determined, the material is added, and the object's geometry is defined. It performs the function of a wrapper over the geometry that is produced (Dekkati et al., 2016). Next, as indicated by the arrow pointing in the direction depicted, the geometry and the material are combined into a form known as mesh, which is then visualized when added to the 3D scene produced within the HTML canvas. This mesh's appearance can be improved visually by including color properties or any texture in the material part of the mesh (Ballamudi, 2019a). Additional light sources have been added to the scene to enhance the visuals or achieve any lighting animation effect that may be necessary. The Three.js package comes with a comprehensive collection of predefined functions, which may be used to generate fundamental 3D shapes based on conventional geometry (Christen & Nebiker, 2015).

This modeling technique is utilized in the construction of the PUMA 560 robot model and is also used for the simulation and kinematic analysis of the robot. When creating the virtual 3D model of the PUMA 560 robot, each 3D component is put together in the correct order and constructed appropriately (Bodepudi et al., 2021). In addition to the direct implication of preset shapes, it is possible to derive specific constructive solid geometry from the essential union and intersection principle of two shapes (Lal, 2016). Developing a virtual model of another industrial robot called the RM501 Movemaster with 5 degrees of freedom uses this methodology of creating a 3D model from an assembly of specified geometry (Desamsetti, 2016a). This model is included in this article as an extra source of information that exemplifies the potential of the modeling approach that is being discussed. As was mentioned before, the Movemaster robot is also put to use in the classroom to instruct students in the field of robot kinematics (Gutlapalli et al., 2019).

MODELING USING A CAD FILE IMPORT PROCEDURE

The procedure can create any three-dimensional virtual model with more straightforward geometry and fundamental design concepts (Desamsetti, 2018). For the most part, using some CAD-creating software for part design is preferable when working with intricate components. After that, the CAD file is imported into the computer so that an accurate model may be created and visualized. The direct use of the WebGL program is hard because all of the gathered data points from the CAD file must be given consecutively to achieve an exact likeness (Thaduri, 2019). Consequently, using the library makes importing CAD files significantly less complicated and time-consuming. The IIT Kharagpur KGP50 robot simulation model is constructed to demonstrate the modeling procedure because it consists of several components with intricate shapes (Lal et al., 2018). Following the completion of the initial design of each robot component in the SolidWorks software, the program script will proceed to prepare the interface for the import of CAD files. The designed components are saved in the stereolithography (STL) format, and then the Three.js STL loader module is used to load them into the software (Lal, 2015).

As previously said, the first steps in constructing the HTML canvas and the 3D scene are the same. The first step is to design the component in Solid Works or another program and then convert the part file into STL format to import it (Koehler et al., 2020). The following stage of the process involves the software calling upon the STL loader library to visualize the geometry. When it comes to 3D modeling, one of the most critical steps is assembling CAD files that have been imported. Once a precise CAD geometry path has been given in the application, a developer can visualize the geometry within an HTML canvas that has been constructed (Gutlapalli, 2017a). Providing its cartesian coordinates makes it possible to position or rotate it in any part of the three-dimensional scene (Hosen et al., 2021). Now, manually through translation or automatically applying a transformation matrix to the geometry, all part components can be assembled in the correct sequence. Stacking one geometry on top of another mathematically is always advisable to achieve the right design (Ballamudi, 2019b). The finished 3D model of the IIT Kharagpur KGP50 robot, together with the model of the actual robot. In addition to the STL format, the part's design can also be saved in any of the other available formats of Three.js. Some 3D building tools, such as Blender, offer an extension of Three.js for the export of 3D scenes, which enables direct mapping into the web interface (Hosen & Gutlapalli, 2021). Because the complete content is uploaded to the worldwide server for the virtual lab's program, as was stated in the introduction, a user is also given the ability to download the CAD files for use in any other kind of experiment. The level of visualization can be taken to an even higher level by applying various materials or image textures to the virtual model to create a more accurate representation (Chistyakov et al., 2016).

SIMULATION ENVIRONMENT SETUP

After the three-dimensional model of the PUMA 560 robot has been fabricated inside the scene, it is mapped

with several different interface components to conduct a kinematic analysis of the robot or to visualize any motion simulation (Mandapuram et al., 2020). Only the most fundamental features required for the kinematic analysis of the robot have been put into the interface, which is linked with the 3D scene, in this case, so that the demonstration may take place (Ballamudi, 2019c). Depending on the developer's requirements, it can be extended to any desired level, and virtually any functionality can be added to the graphical user interface (GUI) (Thaduri et al., 2016). The virtual model of the robot is included in the 3D scene, and it was created using one of the previously described approaches to 3D modeling (Koehler et al., 2020). After that, the scene is coupled with additional components, such as motion simulation, graph display, and manipulator control, along with an input and output panel to perform kinematic analysis (Mandapuram et al., 2018). Other components, such as information regarding the robot and various visual enhancement procedures, such as magnification and rotation, have also been included in the 3D scene (Ballamudi & Desamsetti, 2017). Accurately tracking the end-effector's motion is possible because the view within the 3D environment may be rotated to any orientation, and the magnification level can be adjusted (Thaduri, 2020). It is also possible to divide the overall canvas view into dual or quad-port views to check the robot's movement from all the different projection angles. The entire content program has been condensed into a single HTML file and written in JavaScript (Ballamudi & Desamsetti, 2017). The HTML file must be opened in a web browser that supports HTML5 for the users.

RENDERING ENVIRONMENT LIGHTING EFFECTS

Pre-filtering both diffuse and specular BRDF textures speeds up environment reflection rendering. Since Lambertian refraction is not view-dependent, it may be precisely estimated before rendering using Monte Carlo integration. The pre-filtered environment map is sampled using the standard vector to approximate diffuse environment lighting during rendering (Thaduri, 2021). Using the view-dependent GGXBRDF for specular reflections makes accurate pre-filtering impossible (Ballamudi, 2020). Karis' split sum approximation yields physically reasonable results with pre-filtering. This is how Karis approximates Monte Carlo integration.

Pre-calculating the left sum is simple now that it is not view-dependent. Pre-calculating different resolutions depending on roughness values and storing them in the mipmap chain of the environment texture enables efficient lookup at render time (Lal & Ballamudi, 2017). All other elements are integrated against a white background in the second sum. It depends on material roughness and$|\cos\ \theta k|$. Thus, we pre-calculate a 2D texture lookup of these two values. At render time, the pre-filtered environment map is sampled using the vector. Material roughness determines the mipmap level. Roughness and texture coordinates ($\cos\ \theta k$) are used to test the other sum from the lookup texture (Voutilainen et al., 2014).

DIFFUSE AND SPECULAR OCCLUSION

We can estimate the illumination in an environment by using lookups that have been pre-altered and taken from the scene's textures. However, this does not consider occlusion caused by other items in the scene that are located close (Gutlapalli, 2017b). Real-time rendering makes it challenging to produce physically accurate occlusion calculations, so these calculations are often approximated by employing screen-space ambient occlusion, which reduces the amount of ambient lighting based on the presence of blockers in screen space (Gutlapalli, 2016a). We modify this technique so that it may be used in a progressive rendering, and we also employ another technique that is quite similar to it to detect specular occlusion (Gutlapalli, 2016b). During the process of rendering the scene, we are going to assume that the depth and normal buffers are accessible (Bodepudi et al., 2019). It is usually sufficient to produce them only once with every change in camera perspective.

Sets of diffuse and specular samples are calculated when frame number i is being processed, so letdiandsibe those. A vital sample is used to build a tangent space vector for each sample, which is then used to generate the matching BRDF. To generate disuse samples, a cosine-weighted sample is taken from within the hemisphere, and then this sample is converted into a view space vector (Mandapuram, 2017b). The view space vector is obtained by first generating a half-vector for specular samples based on the material parameters of the rendered surface and then computing the reflection of the view vector at the half-vector (Gutlapalli, 2017c). V is responsible for determining the direction in which the occlusion is examined for this sample. Its magnitude is proportional to the occlusion distance (Mandapuram, 2017a). With the help of this scaled vector, we can determine the sampling point as p=s+v, where is the position of the shaded point in view space. Just let DP be.

The scene depth texture is sampled by transforming the view-space depth of p.p into normalized device coordinates (Dekkati, 2020). This is then used to test the screen-space depth ds from the scene. Now, if dp is

greater than ds, the analyzed sample is judged to be occluding. This results in the environment illumination from the sampled BRDF not contributing to the overall scene lighting, which darkens the obscured area. The effect that diffuse and specular occlusion have on one another (Ye et al., 2016).

PROGRESSIVE AREA LIGHTING

We assume constant outgoing radiance from a light source ($Ldii(p,\ \omega i)$) to model lights with continual intensity over the area. We can use a few samples per light source in a progressive rendering system, but resources are restricted. The intensity of specular reflections from area light sources is calculated using Karis' (Mandapuram, 2016). Approximation of the most representative point. Consider the direction heading to the usual point and the irradiance arriving at point p from area light source A (p). The most representative point is the area light source closest to the view vector's regular vector reflection. To estimate $EA(p)$, choose A as a point light source and ϕA as the total power of the area light source. Combining these equations yields Progressive rendering of penumbras and shows that repeated sampling of a light source across numerous frames produces realistic results. We utilize this method to approximate physically correct shadowing. $Ldio(p,\ \omega o)\ \approx\ 1nn\tilde{O}k{=}1V(p,\ \omega k)f(\omega o,\ \omega r)\phi A4\pi d2|\cos\ \theta i|$ (10) Shadowmapping is used to approximate visibility by changing the light perspective on each sample based on sample ωk. Thus, the shadow mapping technique occludes some samples in the light penumbra but not others. Thus, the aggregated result correctly partially shadows the penumbra. The visual influence of sample size is seen. Sorting samples from closest to farthest from the light source reduces jittering between intermediate frames (Thodupunori & Gutlapalli, 2018). Progressive rendering begins with low discrepancy samples and gradually evaluates more outer samples (Dekkati, 2021). Instead of shadows rushing between frames, penumbras will appear slowly. Define the number of intermediate frames and samples for each light with index i to generate multiple lights. To ensure accurate depiction, the samples for this light are weighted with $wi{=}mni$.

EVALUATION OF VISUAL RENDERING QUALITY & PERFORMANCE

We compare our rendering results to two Blender renderers to confirm our methods. Cycles is a reference renderer for photo-realistic comparisons, and Eevee is a contemporary rasterizer. SketchFab's progressive renderer and Unity and Unreal Engine's physically-based WebGL renderers were compared. They lack lighting support, notably area lights (Demin et al., 2013).

Cornell box drawings. The path-traced reference indicates extensive indirect lighting, particularly on the back wall and box sides. Two rasterizers lack indirect lighting. The top light creates smooth shadows from the two boxes that the Eevee rendering lacks. In contrast, our renderer matches the path-traced reference's shadow shapes and penumbra sizes. The shadow is less intense than the reference but more noticeable than the Eevee rendering. While our progressive technique cannot record precise light bounces, it effectively manages light intensity and shadows based on the reference. Scene rendering for the kitchen (Desamsetti & Lal, 2019). In Cycles, six area lamps over the kitchen counter provide direct lighting. Multiple lights cast delicate shadows and spread penumbras on the wooden drawers. Eevee renders shadows incorrectly in some areas. Especially the cooking utensils and pan shadows need to be covered. The Eevee representation misrepresents the size of the floor penumbra. Again, our renderer accurately matches shadow size and intensity and produces penumbras that match the reference (without noise).

Skyline renderings. Reflections are high on the coating, glass, and tires. Since the car has four lights on all sides, shadows are only seen underneath it (Chen et al., 2019). Again, Eevee renders shadows incorrectly. Shadows under the automobile are roughly the same form but much weaker than in the reference. Our renderer better captures the reference shadow shape and intensity than Eevee. Occlusion features, like the door handle, are better represented than in the Eevee rendering. The bright side ground reactions need to be included (Thaduri & Lal, 2020).

Testing is conducted on three devices to measure rendering speed:

- Desktop with Intel Core i5-3570K and AMD Radeon HD 7950.
- Intel Core i5-4250U with Intel HD Graphics 5000 in a 2013 MacBook Air.
- Motorola Moto X4 with Qualcomm Snap-dragon 630 and Adreno 508 GPU.

Renderer performance comparison. We evaluate rendering times for all three renderers at 1920x1080 (1080p) and 1280x720 (720p) resolutions. This comparison uses 128 frames for our renderer and 64 samples for Eevee. We utilize 1024 samples for Cycles to reduce noise. The results. Cycles take an order of magnitude longer than rasterizers to trace paths. Our progressive renderer outperforms Eevee in all scenes and resolutions. Our rendering is faster than Eevee with 64 samples, even with intermediate results (Shi et al., 2017).

Different hardware performance comparison. Our renderer runs at 128 frames on all three devices. In some circumstances, rendering in 720p on a laptop and desktop saves time in half. The laptop takes several seconds to provide the final rendering results, especially on 1080p resolution, while desktop rendering is instantaneous (Desamsetti & Mandapuram, 2017). Progressive renderers may tolerate this latency because intermediate results are shown during image calculation. Mid-range smartphones cannot render more complicated scenes at 720p and 128 frames (approximately 30 seconds).Evaluation of Interactivity. The first kitchen scene frame rendered in 0.98, 1.28, and 11.02 milliseconds on PC, laptop, and smartphone when tested at 720p.720p and 1080p desktop and laptop rendering speeds are in the millisecond range. We still render interactively in under 16 ms on smartphones. We found that progressive rendering with workload distribution reduces rendering time for the first frame (Xu et al., 2016).

CONCLUSION

The realm of Web3D is undoubtedly one that is fascinating. Its history has abruptly shifted, transforming from one that was gradual and unchanging in the past into one that is fast-changing and dynamic in the present. It is essential to remember that very immediately after the launch of WebGL, the Web3D market was regarded by software businesses as mature. This resulted in immediate and noticeable investments, which led to the development of several commercial-grade software and hardware solutions within a very brief period. The sudden availability of many valuable tools significantly differs from what occurred in the past with other media. In the past, the web was exploited by trial and error by players of various sizes and riches, ranging from underdogs to significant businesses. This did not occur when excellent tools became suddenly available. This sector's quick expansion has influenced its evolution and development and established de facto standards in file formats, user interface paradigms, and the kinds of functionality currently available. This market-driven character is further shown by the fact that the majority of the tools and systems that are presently available still need a reference scientific publication, even though they propose unique technical solutions.

REFERENCES

Amin, R., & Mandapuram, M. (2021). CMS - Intelligent Machine Translation with Adaptation and AI. *ABC Journal of Advanced Research*, *10*(2), 199-206. https://doi.org/10.18034/abcjar.v10i2.693

Ballamudi, V. K. R. (2016). Utilization of Machine Learning in a Responsible Manner in the Healthcare Sector. *Malaysian Journal of Medical and Biological Research*, *3*(2), 117-122. https://mjmbr.my/index.php/mjmbr/article/view/677

Ballamudi, V. K. R. (2019a). Artificial Intelligence: Implication on Management. *Global Disclosure of Economics and Business*, *8*(2), 105-118. https://doi.org/10.18034/gdeb.v8i2.540

Ballamudi, V. K. R. (2019b). Road Accident Analysis and Prediction using Machine Learning Algorithmic Approaches. *Asian Journal of Humanity, Art and Literature*, *6*(2), 185-192. https://doi.org/10.18034/ajhal.v6i2.529

Ballamudi, V. K. R. (2019c). Hybrid Automata: An Algorithmic Approach Behavioral Hybrid Systems. *Asia Pacific Journal of Energy and Environment*, *6*(2), 83-90. https://doi.org/10.18034/apjee.v6i2.541

Ballamudi, V. K. R. (2020). Militarization of Space. *Asian Journal of Applied Science and Engineering*, *9*(1), 169–178. https://doi.org/10.18034/ajase.v9i1.38

Ballamudi, V. K. R., & Desamsetti, H. (2017). Security and Privacy in Cloud Computing: Challenges and Opportunities. *American Journal of Trade and Policy*, *4*(3), 129–136. https://doi.org/10.18034/ajtp.v4i3.667

Ballamudi, V. K. R., Lal, K., Desamsetti, H., & Dekkati, S. (2021). Getting Started Modern Web Development with Next.js: An Indispensable React Framework. *Digitalization & Sustainability Review*, *1*(1), 1–11. https://upright.pub/index.php/dsr/article/view/83

Bodepudi, A., Reddy, M., Gutlapalli, S. S., & Mandapuram, M. (2019). Voice Recognition Systems in the Cloud Networks: Has It Reached Its Full Potential?. *Asian Journal of Applied Science and Engineering*, *8*(1), 51–60. https://doi.org/10.18034/ajase.v8i1.12

Bodepudi, A., Reddy, M., Gutlapalli, S. S., & Mandapuram, M. (2021). Algorithm Policy for the Authentication of Indirect Fingerprints Used in Cloud Computing. *American Journal of Trade and Policy*, *8*(3), 231–238. https://doi.org/10.18034/ajtp.v8i3.651

Chen, S., Thaduri, U. R., & Ballamudi, V. K. R. (2019). Front-End Development in React: An Overview. *Engineering International*, *7*(2), 117–126. https://doi.org/10.18034/ei.v7i2.662

Chistyakov, A., Soto, M. T., Martí, E., Carrabina, J. (2016). An HTML Tool for Production of Interactive Stereoscopic Compositions. *Journal of Medical Systems*, *40*(12), 1-9. https://doi.org/10.1007/s10916-016-0616-0

Christen, M., Nebiker, S. (2015). Visualisation of Complex 3D City Models on Mobile Web browsers Using Cloud-Based Image Provisioning. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, *II-3/W5*, 517-522. https://doi.org/10.5194/isprsannals-II-3-W5-517-2015

Dekkati, S. (2020). Blender and Unreal Engine Character Design and Behavior Programming for 3D Games. *ABC Journal of Advanced Research*, *9*(2), 115-126. https://doi.org/10.18034/abcjar.v9i2.704

Dekkati, S. (2021). Python Programming Language for Data-Driven Web Applications. *International Journal of Reciprocal Symmetry and Theoretical Physics*, *8*, 1–10. https://upright.pub/index.php/ijrstp/article/view/90

Dekkati, S., & Thaduri, U. R. (2017). Innovative Method for the Prediction of Software Defects Based on Class Imbalance Datasets. *Technology & Management Review*, *2*, 1–5. https://upright.pub/index.php/tmr/article/view/78

Dekkati, S., Lal, K., & Desamsetti, H. (2019). React Native for Android: Cross-Platform Mobile Application Development. *Global Disclosure of Economics and Business*, *8*(2), 153-164. https://doi.org/10.18034/gdeb.v8i2.696

Dekkati, S., Thaduri, U. R., & Lal, K. (2016). Business Value of Digitization: Curse or Blessing?. *Global Disclosure of Economics and Business*, *5*(2), 133-138. https://doi.org/10.18034/gdeb.v5i2.702

Demin, A., Zhang, T. R., Geng, H. S., Li, X., Wang, W. S. (2013). Research on Motion Simulation for Robot Based on WebGL. *Applied Mechanics and Materials*, *341-342*, 704. https://doi.org/10.4028/www.scientific.net/AMM.341-342.704

Deming, C., Dekkati, S., & Desamsetti, H. (2018). Exploratory Data Analysis and Visualization for Business Analytics. *Asian Journal of Applied Science and Engineering*, *7*(1), 93–100. https://doi.org/10.18034/ajase.v7i1.53

Desamsetti, H. (2016a). A Fused Homomorphic Encryption Technique to Increase Secure Data Storage in Cloud Based Systems. *The International Journal of Science & Technoledge*, *4*(10), 151-155.

Desamsetti, H. (2016b). Issues with the Cloud Computing Technology. *International Research Journal of Engineering and Technology (IRJET)*, *3*(5), 321-323.

Desamsetti, H. (2018). Internet of Things (IoT) Technology for Use as Part of the Development of Smart Home Systems. *International Journal of Reciprocal Symmetry and Theoretical Physics*, *5*, 14–21. https://upright.pub/index.php/ijrstp/article/view/89

Desamsetti, H. (2020). Relational Database Management Systems in Business and Organization Strategies. *Global Disclosure of Economics and Business*, *9*(2), 151-162. https://doi.org/10.18034/gdeb.v9i2.700

Desamsetti, H. (2021). Crime and Cybersecurity as Advanced Persistent Threat: A Constant E-Commerce Challenges. *American Journal of Trade and Policy*, *8*(3), 239–246. https://doi.org/10.18034/ajtp.v8i3.666

Desamsetti, H., & Lal, K. (2019). Being a Realistic Master: Creating Props and Environments Design for AAA Games. *Asian Journal of Humanity, Art and Literature*, *6*(2), 193-202. https://doi.org/10.18034/ajhal.v6i2.701

Desamsetti, H., & Mandapuram, M. (2017). A Review of Meta-Model Designed for the Model-Based Testing Technique. *Engineering International*, *5*(2), 107–110. https://doi.org/10.18034/ei.v5i2.661

Gutlapalli, S. S. (2016a). An Examination of Nanotechnology's Role as an Integral Part of Electronics. *ABC Research Alert*, *4*(3), 21–27. https://doi.org/10.18034/ra.v4i3.651

Gutlapalli, S. S. (2016b). Commercial Applications of Blockchain and Distributed Ledger Technology. *Engineering International*, *4*(2), 89–94. https://doi.org/10.18034/ei.v4i2.653

Gutlapalli, S. S. (2017a). Analysis of Multimodal Data Using Deep Learning and Machine Learning. *Asian Journal of Humanity, Art and Literature*, *4*(2), 171–176. https://doi.org/10.18034/ajhal.v4i2.658

Gutlapalli, S. S. (2017b). The Role of Deep Learning in the Fourth Industrial Revolution: A Digital Transformation Approach. *Asian Accounting and Auditing Advancement*, *8*(1), 52–56. https://4ajournal.com/article/view/77

Gutlapalli, S. S. (2017c). An Early Cautionary Scan of the Security Risks of the Internet of Things. *Asian Journal of Applied Science and Engineering*, *6*, 163–168. Retrieved from https://ajase.net/article/view/14

Gutlapalli, S. S., Mandapuram, M., Reddy, M., & Bodepudi, A. (2019). Evaluation of Hospital Information Systems (HIS) in terms of their Suitability for Tasks. *Malaysian Journal of Medical and Biological Research*, *6*(2), 143–150. https://mjmbr.my/index.php/mjmbr/article/view/661

Hosen, M. S., & Gutlapalli, S. S. (2021). A Study of Innovative Class Imbalance Dataset Software Defect Prediction Methods. *Asian Journal of Applied Science and Engineering*, *10*(1), 52–55. https://doi.org/10.18034/ajase.v10i1.52

Hosen, M. S., Thaduri, U. R., Ballamudi, V. K. R., & Lal, K. (2021). Photo-Realistic 3D Models and Animations for Video Games and Films. *Engineering International*, *9*(2), 153–164. https://doi.org/10.18034/ei.v9i2.668

Kang, S., Lee, J. (2017). Developing a Tile-Based Rendering Method to Improve Rendering Speed of 3D Geospatial Data with HTML5 and WebGL. *Journal of Sensors*, *2017*, https://doi.org/10.1155/2017/9781307

Koehler, S., Desamsetti, H., Ballamudi, V. K. R., & Dekkati, S. (2020). Real World Applications of Cloud Computing: Architecture, Reasons for Using, and Challenges. *Asia Pacific Journal of Energy and Environment*, *7*(2), 93-102. https://doi.org/10.18034/apjee.v7i2.698

Lal, K. (2015). How Does Cloud Infrastructure Work?. *Asia Pacific Journal of Energy and Environment*, *2*(2), 61-64. https://doi.org/10.18034/apjee.v2i2.697

Lal, K. (2016). Impact of Multi-Cloud Infrastructure on Business Organizations to Use Cloud Platforms to Fulfill Their Cloud Needs. *American Journal of Trade and Policy*, *3*(3), 121–126. https://doi.org/10.18034/ajtp.v3i3.663

Lal, K., & Ballamudi, V. K. R. (2017). Unlock Data's Full Potential with Segment: A Cloud Data Integration Approach. *Technology & Management Review*, *2*(1), 6–12. https://upright.pub/index.php/tmr/article/view/80

Lal, K., Ballamudi, V. K. R., & Thaduri, U. R. (2018). Exploiting the Potential of Artificial Intelligence in Decision Support Systems. *ABC Journal of Advanced Research*, *7*(2), 131-138. https://doi.org/10.18034/abcjar.v7i2.695

Mandapuram, M. (2016). Applications of Blockchain and Distributed Ledger Technology (DLT) in Commercial Settings. *Asian Accounting and Auditing Advancement*, *7*(1), 50–57. https://4ajournal.com/article/view/76

Mandapuram, M. (2017a). Application of Artificial Intelligence in Contemporary Business: An Analysis for Content Management System Optimization. *Asian Business Review*, *7*(3), 117–122. https://doi.org/10.18034/abr.v7i3.650

Mandapuram, M. (2017b). Security Risk Analysis of the Internet of Things: An Early Cautionary Scan. *ABC Research Alert*, *5*(3), 49–55. https://doi.org/10.18034/ra.v5i3.650

Mandapuram, M., & Hosen, M. F. (2018). The Object-Oriented Database Management System versus the Relational Database Management System: A Comparison. *Global Disclosure of Economics and Business*, *7*(2), 89–96. https://doi.org/10.18034/gdeb.v7i2.657

Mandapuram, M., Gutlapalli, S. S., Bodepudi, A., & Reddy, M. (2018). Investigating the Prospects of Generative Artificial Intelligence. *Asian Journal of Humanity, Art and Literature*, *5*(2), 167–174. https://doi.org/10.18034/ajhal.v5i2.659

Mandapuram, M., Gutlapalli, S. S., Reddy, M., Bodepudi, A. (2020). Application of Artificial Intelligence (AI) Technologies to Accelerate Market Segmentation. *Global Disclosure of Economics and Business* *9*(2), 141–150. https://doi.org/10.18034/gdeb.v9i2.662

Reddy, M., Bodepudi, A., Mandapuram, M., & Gutlapalli, S. S. (2020). Face Detection and Recognition Techniques through the Cloud Network: An Exploratory Study. *ABC Journal of Advanced Research*, *9*(2), 103–114. https://doi.org/10.18034/abcjar.v9i2.660

Sawicki, B., Chaber, B. (2013). Efficient Visualization of 3D Models by Web Browser. *Computing. Archives for Informatics and Numerical Computation, suppl. Supplement*, *95*, 661-673. https://doi.org/10.1007/s00607-012-0275-z

Shi, X., Fu, J., Jin, C. (2017). Development of 3D Browsing and Interactive Web System. *IOP Conference Series. Materials Science and Engineering*, *242*(1). https://doi.org/10.1088/1757-899X/242/1/012113

Sun, F., Zhang, Z., Liao, D., Chen, T., Zhou, J. (2015). A Lightweight and Cross-Platform Web3D System for Casting Process Based on Virtual Reality Technology Using WebGL. *The International Journal of Advanced Manufacturing Technology*, *80*(5-8), 801-816. https://doi.org/10.1007/s00170-015-7050-1

Thaduri, U. R. (2017). Business Security Threat Overview Using IT and Business Intelligence. *Global Disclosure of Economics and Business*, *6*(2), 123-132. https://doi.org/10.18034/gdeb.v6i2.703

Thaduri, U. R. (2018). Business Insights of Artificial Intelligence and the Future of Humans. *American Journal of Trade and Policy*, *5*(3), 143–150. https://doi.org/10.18034/ajtp.v5i3.669

Thaduri, U. R. (2019). Android & iOS Health Apps for Track Physical Activity and Healthcare. *Malaysian Journal of Medical and Biological Research*, *6*(2), 151-156. https://mjmbr.my/index.php/mjmbr/article/view/678

Thaduri, U. R. (2020). Decision Intelligence in Business: A Tool for Quick and Accurate Marketing Analysis. *Asian Business Review*, *10*(3), 193–200. https://doi.org/10.18034/abr.v10i3.670

Thaduri, U. R. (2021). Virtual Reality & Artificial Intelligence in Real Estate Business: A Tool for Effective Marketing Campaigns. *Asian Journal of Applied Science and Engineering*, *10*(1), 56–65. https://doi.org/10.18034/ajase.v10i1.54

Thaduri, U. R., & Lal, K. (2020). Making a Dynamic Website: A Simple JavaScript Guide. *Technology & Management Review*, *5*, 15–27. https://upright.pub/index.php/tmr/article/view/81

Thaduri, U. R., Ballamudi, V. K. R., Dekkati, S., & Mandapuram, M. (2016). Making the Cloud Adoption Decisions: Gaining Advantages from Taking an Integrated Approach. *International Journal of Reciprocal Symmetry and Theoretical Physics*, *3*, 11–16. https://upright.pub/index.php/ijrstp/article/view/77

Thodupunori, S. R., & Gutlapalli, S. S. (2018). Overview of LeOra Software: A Statistical Tool for Decision Makers. *Technology & Management Review*, *3*(1), 7–11.

Voutilainen, J-P., Mattila, A-L., Mikkonen, T. (2014). Lively3D: Building a 3D Desktop Environment as a Single Page Application. *Acta Cybernetica*, *21*(3), 291-306. https://doi.org/10.14232/actacyb.21.3.2014.2

Xu, Z., Zhang, Y., Xu, X. (2016). 3D Visualization for Building Information Models Based Upon IFC and WebGL Integration. *Multimedia Tools and Applications*, *75*(24), 17421-17441. https://doi.org/10.1007/s11042-016-4104-9

Ye, M., Wei, S., Zhang, D. (2016). An Approach of Web-based Point Cloud Visualization without Plugin. *IOP Conference Series. Earth and Environmental Science*, *46*(1). https://doi.org/10.1088/1755-1315/46/1/012011

--0--